

Preprocessing Task

Uber Fare Prediction project

Prepared by : Ammar Mohamed



Table of contents

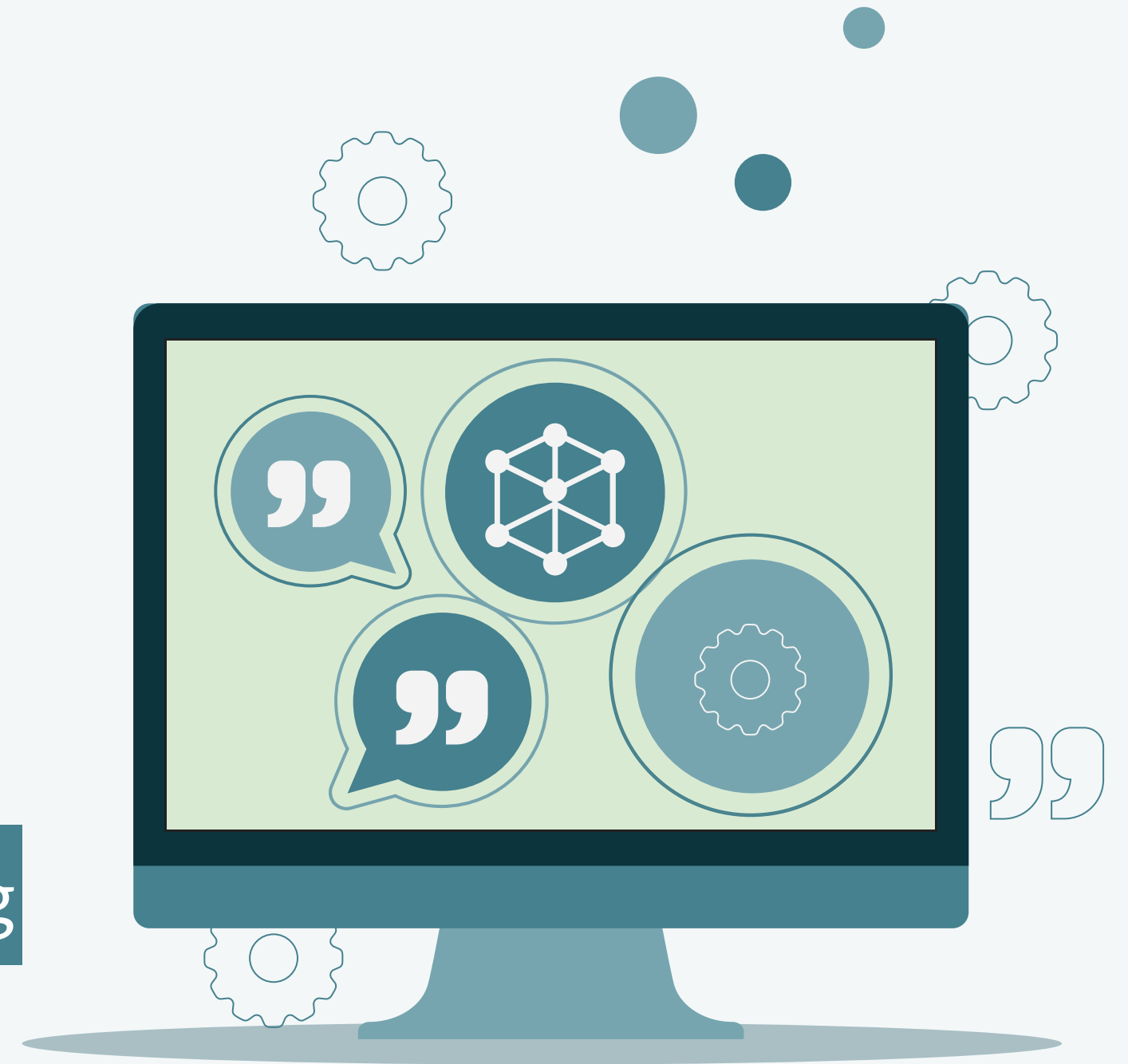
01 Preprocessing Decisions

02 Pipeline & Model

03 Results & Next Steps

01 Preprocessing Decisions

Handling Outliers + Feature Eng + data splitting



Handling Outliers Manually

```
[13]: # 1. Count how many rows are outside the [2.5, 50] range
outliers_under = df[df['fare_amount'] < 2.5]
outliers_over = df[df['fare_amount'] > 50]

total_rows = len(df)
num_under = len(outliers_under)
num_over = len(outliers_over)
total_outliers = num_under + num_over

print(f"Total Rows in Dataset: {total_rows:,}\n")
print(f"Fares under $2.5:      {num_under:,} ({num_under/total_rows:.4%})")
print(f"Fares over $50:        {num_over:,} ({num_over/total_rows:.4%})")
print(f"Total violations:       {total_outliers:,} ({total_outliers/total_rows:.4%})")
```

Total Rows in Dataset: 482,519

Fares under \$2.5: 0 (0.0000%)
Fares over \$50: 5,948 (1.2327%)
Total violations: 5,948 (1.2327%)

```
[11]: # checking fare outliers
df['fare_amount'].describe()
```

```
[11]: count    482519.000000
      mean      11.339797
      std       9.556656
      min       2.500000
      25%       6.000000
      50%       8.500000
      75%      12.500000
      max      199.500000
      Name: fare_amount, dtype: float64
```

```
[12]: # checking distance outliers
df['distance'].describe()
```

```
[12]: count    482519.000000
      mean      3.372081
      std       3.786587
      min      0.000084
      25%       1.280582
      50%       2.179000
      75%       3.958661
      max      110.833077
      Name: distance, dtype: float64
```

Chronological ordering and split

We used a chronological train-test split to simulate a real-world deployment scenario. The model is trained on historical taxi rides and evaluated on future rides it has never seen before. This reflects how the system would operate in production, where predictions are always made on new incoming data.

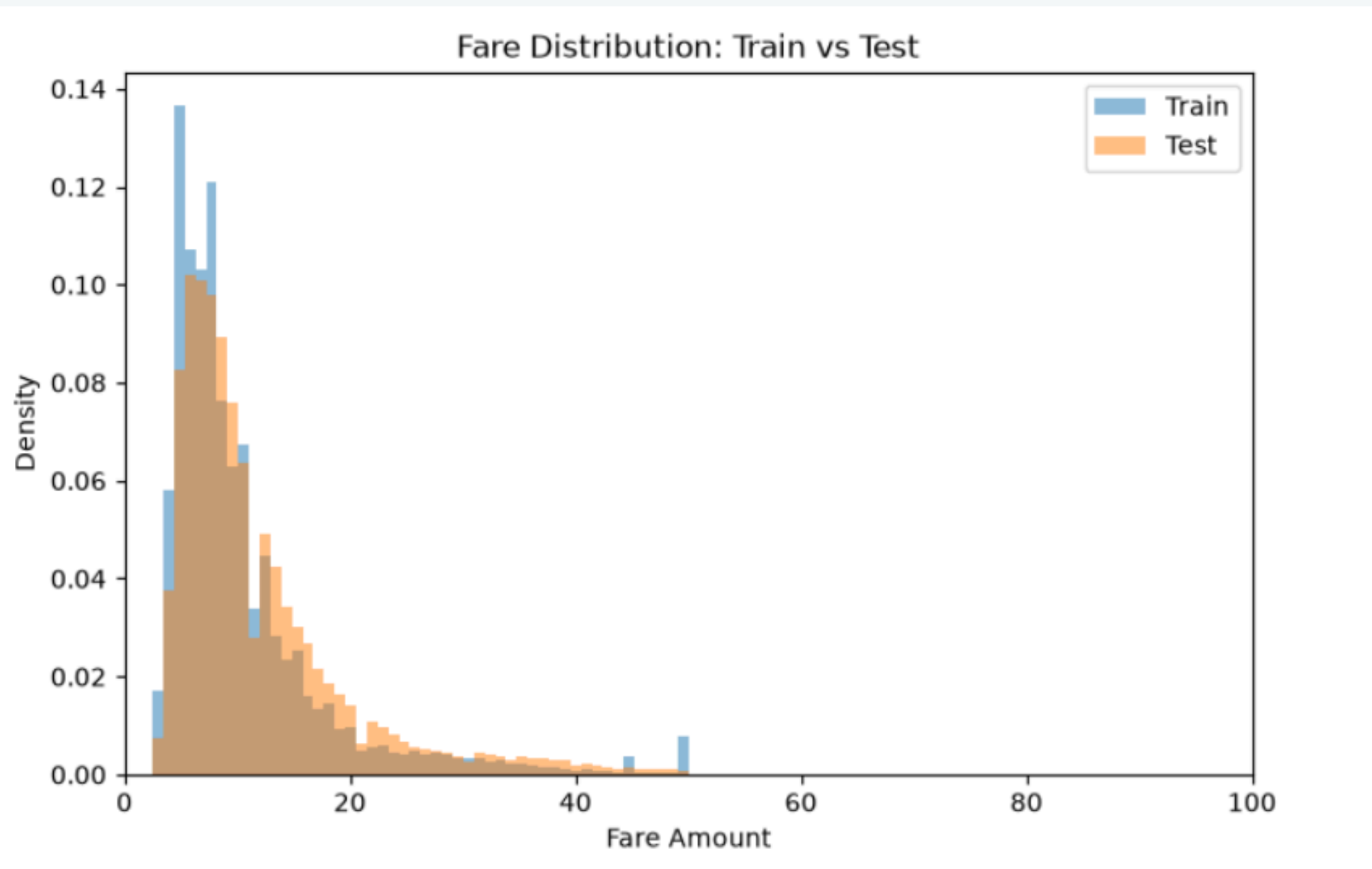
```
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])
df = df.sort_values('pickup_datetime').reset_index(drop=True)

# 2. Define target and features from the sorted DataFrame
y = df['fare_amount']
X = df.drop(columns=['fare_amount'])

# 3. Calculate split index for the last 20% of data
split_idx = int(len(df) * 0.8)

# Chronological Split (No random shuffling!)
X_train = X.iloc[:split_idx]
X_test = X.iloc[split_idx:]
y_train = y.iloc[:split_idx]
y_test = y.iloc[split_idx:]

print("Training set shape (earlier 80%):", X_train.shape)
print("Test set shape (latest 20%):", X_test.shape)
print(f"Training times: {X_train['pickup_datetime'].min()} to {X_train['pickup_datetime'].max()}")
print(f"Testing times: {X_test['pickup_datetime'].min()} to {X_test['pickup_datetime'].max()}")
```



This Chart shows that :

even after spliting according to year we maintian balance between train and test data wrt fair amount which protects our model from overfitting to a specific time period and failing to generalize when predicting fares from a different range of dates.

Feature Selection

Dropped Columns :

```
# NOW drop the unnecessary/redundant columns – applied to both train and test separately
columns_to_drop = [
    'pickup_datetime', 'season', 'day',
    'sol_dist',
    'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude', 'is_holiday', 'bearing'
]

X_train_selected = X_train.drop(columns=columns_to_drop, errors='ignore')
X_test_selected = X_test.drop(columns=columns_to_drop, errors='ignore')
```

```
Final selected training columns: ['Car Condition', 'Weather', 'Traffic Condition', 'passenger_count', 'month', 'weekday', 'year', 'jfk_dist', 'ewr_dist', 'lga_dist', 'nyc_dist', 'distance', 'hour_sin', 'hour_cos']
Processed training set shape: (381190, 14)
```

Feature Engineering

Hour was converted from a single 0-23 number into two features (hour_sin, hour_cos) that represent it as a position on a 24-hour circle — this correctly tells the model that 11 PM and midnight are neighbors in time, not opposites.

```
# Feature Engineering: converting hours into cyclic features
for df_part in [X_train_selected, X_test_selected]:
    df_part['hour_sin'] = np.sin(2 * np.pi * df_part['hour'] / 24)
    df_part['hour_cos'] = np.cos(2 * np.pi * df_part['hour'] / 24)
```

02 Pipeline & Model

Encoding + Scaling + Tuning

—Someone Famous



Pipeline Process

```
[24]: from sklearn.pipeline import Pipeline
      from sklearn.compose import ColumnTransformer
      from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler

      # Define column groups
      ordinal_col = ['Car Condition']
      ordinal_order = [['Bad', 'Good', 'Very Good', 'Excellent']]
      onehot_cols = ['Weather', 'Traffic Condition']
      numeric_cols = ['passenger_count', 'weekday', 'month', 'year',
                      'jfk_dist', 'ewr_dist', 'lga_dist', 'nyc_dist',
                      'distance']

      # Numeric branch: scale only
      numeric_pipeline = Pipeline(steps=[
          ('scaler', StandardScaler())
      ])

      # Categorical branches
      ordinal_pipeline = Pipeline(steps=[
          ('ordinal', OrdinalEncoder(categories=ordinal_order))
      ])

      onehot_pipeline = Pipeline(steps=[
          ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore'))
      ])

      # Combine all branches
      preprocessor = ColumnTransformer(transformers=[
          ('num', numeric_pipeline, numeric_cols),
          ('ord', ordinal_pipeline, ordinal_col),
          ('onehot', onehot_pipeline, onehot_cols)
      ], remainder='passthrough')
```

```
[25]: full_pipeline = Pipeline(steps=[
      ('preprocessor', preprocessor),
      ('model', LinearRegression())
  ])

  # One single call does imputation, encoding, scaling, AND fits the model
  full_pipeline.fit(X_train_selected, y_train)
```

HyperParameter Tuning

- Down-sample chronologically — take every 10th row (10% of training data) to speed up tuning, while preserving the full time range from 2009–2014
- Time-aware cross-validation — use TimeSeriesSplit (5 folds) instead of random K-Fold, so each fold only trains on data before its validation period, respecting real-world chronology
- Build a unified pipeline — bundle preprocessing + Gradient Boosting into a single Pipeline object, ensuring consistent, leak-free transformations at every step
- Define the search space — specify candidate values for 5 key hyperparameters (n_estimators, learning_rate, max_depth, min_samples_split, subsample)
- Randomized search for best parameters — test 10 random combinations across all 5 time-based folds (50 total fits), scored by MAE, to efficiently find strong hyperparameters without exhaustive search

```
Original training size: 381,190
Tuning sample size:      38,119 (Chronological order preserved)

Starting fast Gradient Boosting tuning with TimeSeriesSplit...
Tuning complete!
Best Gradient Boosting Parameters:
{'model__subsample': 0.9, 'model__n_estimators': 200, 'model__min_samples_split': 5, 'model__max_depth': 4, 'model__learning_rate': 0.1}
```

Model Evaluation

How Do We Judge Model Performance?

We consider three key metrics:

- MAE (Mean Absolute Error) – Measures the average prediction error.
- RMSE (Root Mean Squared Error) – Penalizes large prediction errors more heavily.
- R^2 Score (Coefficient of Determination) – Measures how well the model explains the variation in the data.

that's means :

Lower **MAE** → More accurate predictions.

Lower **RMSE** → Fewer large prediction errors.

Higher **R^2** → Better ability to explain and predict the target variable.

Model Evaluation

```
# Build a fresh pipeline using the optimal parameters
final_gb_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('model', GradientBoostingRegressor(**best_params, random_state=42))
])
```

```
# Fit on 100% of the data
final_gb_pipeline.fit(X_train_selected, y_train)
```

```
# 3. Evaluate your baseline model (from full_pipeline)
evaluate_pipeline(full_pipeline, X_test_selected, y_test, name="Baseline Linear Regression")
```

```
# 4. Evaluate your fully trained, tuned Gradient Boosting model
evaluate_pipeline(final_gb_pipeline, X_test_selected, y_test, name="Tuned Gradient Boosting (Full Train)")
```

```
Rebuilding and training the optimal Gradient Boosting model on the FULL training set...
Training on full dataset complete!
```

```
=== Baseline Linear Regression Performance on Test Set ===
```

```
MAE (Mean Absolute Error): 2.3762
RMSE (Root Mean Squared Error): 3.6162
R2 (Coefficient of Det.): 0.7901
```

```
=== Tuned Gradient Boosting (Full Train) Performance on Test Set ===
```

```
MAE (Mean Absolute Error): 1.8483
RMSE (Root Mean Squared Error): 2.9209
R2 (Coefficient of Det.): 0.8631
```

```
{'MAE': 1.8482996771837137,
 'RMSE': np.float64(2.9208676059275276),
 'R2': 0.8630719754119986}
```

Model Evaluation

As we see the model performance is 86 which is not bad remember this number well because there is a surprise coming in our way

To be honest the first time I run the past cells the efficiency was 75% only

yeah , but with some modification we go up to 86 like:

- Handling outliers manually.
- Switching from elastic Net to Gradient boost regressor.
- Splitting the data Chronologically .

03 Results & Next Steps

The Optimized Notebook is here



Best Practices and Enhancements

Before leaving the old notebook we asked a very valuable question:

Does ignoring over half our features actually affects accuracy?

This insight will make a big difference in the new optimized notebook we will navigate together right now

but we can summarize the difference between them in :

- Pipeline include all the preprocessing .
- Outliers are auto checked using Class.
- Multicollinearity is auto checked using VIF.
- We trained only the most imp features
- No need for Encoding

	feature	importance
	num__distance	0.918668
	num__lga_dist	0.022922
	num__jfk_dist	0.021727
	num__year	0.017785
	remainder__hour_cos	0.004987
	num__ewr_dist	0.004063
	num__nyc_dist	0.003446
	num__month	0.002696
	remainder__hour_sin	0.001624
	num__weekday	0.001547
	num__passenger_count	0.000216
	onehot__Weather_stormy	0.000101
onehot__Traffic	Condition_Dense Traffic	0.000071
	onehot__Weather_sunny	0.000054
	ord__Car Condition	0.000032
	onehot__Weather_windy	0.000031
onehot__Traffic	Condition_Flow Traffic	0.000016
	onehot__Weather_rainy	0.000014

Conclusion

```
from sklearn.linear_model import LinearRegression

baseline_pipeline = Pipeline(steps=[
    ('outlier_capping', OutlierCapper(lower_pct=0.01, upper_pct=0.99)),
    ('vif_filter', VIFDropper(threshold=10.0)),
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

baseline_pipeline.fit(X_train, y_train)
y_pred_baseline = baseline_pipeline.predict(X_test)

print("Baseline MAE:", mean_absolute_error(y_test, y_pred_baseline))
print("Baseline RMSE:", np.sqrt(mean_squared_error(y_test, y_pred_baseline)))
print("Baseline R²:", r2_score(y_test, y_pred_baseline))
```

Baseline MAE: 2.694782343688186
Baseline RMSE: 5.10050725715256
Baseline R²: 0.7879960391304457

```
full_pipeline = Pipeline(steps=[
    ('outlier_capping', OutlierCapper(lower_pct=0.01, upper_pct=0.99, exclude_cols=['year'])),
    ('vif_filter', VIFDropper(threshold=10.0)),
    ('scaler', StandardScaler()),
    ('model', HistGradientBoostingRegressor(random_state=42))
])
```

MAE: 2.1122
RMSE: 4.0049
R²: 0.8693

Thanks!

Do you have any questions?



<https://github.com/ammaraabdelghany30/uber-data-analysis>